生成 AI による教材開発の民主化とその課題〈実務編〉

― デプロイ環境の仕組みとリスク、API・利用規約、保護者対応の整理 ―

二部構成の〈実務編〉。背景と論旨は〈論考編〉に、実務的な手順・チェックリストは本稿にまとめている。

飯綱町立飯綱中学校　髙野 健人

1. 本編の位置づけとねらい

本稿は、姉妹編〈論考編〉「生成 AI による教材開発の民主化とその課題」と対をなす〈実務編〉である。〈論考編〉では、生成 AI によって教材開発への参加の条件が急速に整いつつある一方、「作れるか」だけでなく「安全に作れるか・適切に運用できるか」が問われることを論じた。本〈実務編〉は、その「安全に・適切に」を、現場で使える形に具体化する。すなわち、（一）自作した教材を公開・配布する主な環境の仕組みと危うさ、そして情報流出を防ぐ手立て、（二）生徒が使う教材での API 利用と、学校で Azure が有力とされる事情、（三）混同されがちな「生成 AI の利用制限」と「API で作った教材（作品）の制限」の違い、（四）見落とされがちな保護者への通知・事前指導を、順に整理する。作ることが容易になった今こそ、これらが後回しになっていないかを点検したい。

2. コードを公開・デプロイする主な環境 ― 仕組み・危うさ・対策

自作教材は、どこかに「置いて公開」して初めて生徒が使える。どこに置くかによって、守り方が変わる。まず、環境を問わず共通する原則を挙げる。

- 秘密の鍵（API キー、サービスアカウント鍵、パスワード）は、ソースコードに直接書かない。環境変数やシークレット管理機能に置き、リポジトリには含めない。
- 公開範囲は最小に。「URL を知っていれば誰でも見られる」を避け、必要に応じてログインや合言葉（PIN）を設ける。
- 「誰が読み書きできるか」という権限は、ブラウザ側ではなくサーバー側で担保する。ブラウザに送られたものは、すべて見られる前提で考える。
- 生徒の個人情報は「集めない・分ける・消す」。必要最小限にとどめ、保存先と権限を決め、保持期間（自動削除）まで設計する。
- 万一鍵が漏れたら、最優先は「鍵の無効化・再発行（ローテート）」。コードから消すだけでは、

履歴に残った鍵は使われ続けうる。

表 A　主なデプロイ環境の仕組み・危うさ・対策

環境	仕組み	主な危うさ	流出を防ぐ要点
GitHub	ソースコードの保管・共有。多くの無料公開の起点となる。	鍵を誤ってコミットすると公開露出。履歴に残るため、最新から消しても過去に残る。	鍵は .gitignore で除外。シークレットスキャン／プッシュ保護を有効に。漏れたら鍵を再発行し、履歴も削除。
Netlify・Vercel	HTML／JS／CSS を配信する静的ホスティング。動的処理はサーバーレス関数で行う。	ブラウザに配られる JS や、公開ビルドに埋め込まれる変数に書いた鍵は、そのまま見える。	秘匿鍵はクライアントに出さない。鍵を使う処理はサーバーレス関数側に置き、ブラウザはそれを呼ぶだけにする。
Streamlit Cloud	GitHub のコードから Web アプリを自動公開。生成される URL は既定で誰でもアクセス可（コードはサーバー側で実行）。	secrets ファイルの誤コミット。URL 公開のため、対策がないと誰でも利用できる。	秘密は Cloud の「Secrets」欄に置き st.secrets で参照。必要ならアプリにログイン等を実装。コードには鍵を含めない。
Firebase	ホスティング＋データベース（Firestore）＋認証。ブラウザから直接 DB に読み書きできるのが特徴。	「セキュリティルール」未設定（テストモード）で公開すると、公開用キーで誰でも読み書きでき、個人情報が漏れる。	本番／ロックモードで「既定は全拒否」から始め、必要な操作だけ許可。App Check や監視を併用。公開前にルールを検証。
GAS Web アプリ	スプレッドシート等と連携するスクリプトを Web 公開。「実行するユーザー」「アクセスできるユーザー」を選ぶ。	「自分として実行 × 全員アクセス」にすると、URL を知る誰もが所有者の権限でスクリプトを動かせる（所有者のシート等に到達）。	公開範囲は必要最小限に。全員公開時は所有者権限で動かさない設計に。鍵はスクリプトプロパティに置く。

特に誤解されやすい点を補う。第一に、Firebase の「API キー」は、そもそも公開される前提の識別子であり、それ自体は秘密ではない。守りの本体は「セキュリティルール」であって、ルールが緩ければ、鍵を隠していても情報は漏れる。逆に、鍵が見えていてもルールが正しければ守られる。第二に、静的ホスティング（Netlify 等）では、「環境変数に入れたから安全」とは限らない。ビルド時にブラウザ側へ埋め込まれる種類の変数は露出する。消費者向け AI の API キーをフロントに置くと、第三者に不正利用される。第三に、GAS では、コードはサーバー側

で動くため鍵はブラウザに出ないが、スクリプトやシートを共有・コピーすると鍵も一緒に渡る点に注意する。

3. 生徒が使う教材での API 利用 — なぜ「Azure」が有力か

生徒に使わせる自作教材で生成 AI を呼び出す場合、どの API を使うかは、技術以前に「規約と承認」の問題である。まず、消費者向けサービスには、各社が規約で年齢制限を設けている（2026 年時点。学校配布アカウントでの扱いは別に記す）。

表 B　主な生成 AI サービスの年齢制限（各社が規約で設定）

サービス	利用可能年齢（規約）	学校での扱い（目安）
ChatGPT（OpenAI）	13 歳以上。18 歳未満は保護者の同意が必要（全モデル・全プラン共通）。	直接利用には年齢と保護者同意の確認が要る。
Gemini（Google）	一般は 13 歳以上（2024 年 8 月に引き下げ）。	学校用（Workspace／Education）アカウントは 18 歳以上とされる。
Copilot（Microsoft）	日本では個人アカウントは実質 18 歳以上が対象。	学校の Microsoft 365 アカウントでは、2025 年半ば以降 Copilot Chat 等が 13 歳以上に提供（データ保護付き）。
Claude（Anthropic）	18 歳以上。	直接利用は 18 歳以上。

※背景には、米国の COPPA（13 歳未満の個人情報保護）や EU の GDPR 等がある。本人確認は厳密でなく「使えてしまう」が、規約違反はアカウント停止等のリスクを伴う。

ここで決定的に重要なのは、これらの年齢制限は「サービスを直接使う人」に対して各社が課すものだという点である。教師が Azure OpenAI Service のような開発者向けの基盤（API）を使って教材を作る場合、年齢や利用範囲の制限は、開発者すなわち教師・学校が設定する。生徒は教師の作ったアプリを使うのであって、その AI サービスに自分のアカウントを持つわけではない。

Azure OpenAI Service は、OpenAI のモデルを Microsoft Azure 上で利用できる基盤であり、入力したデータはモデルの学習に使われず、自組織の管理下（テナント内）にとどまる。コンテンツフィルタやアクセス制御も開発側で設定できる。データ保護・開発者による制御・GIGA 端末で普及する Microsoft 365 との親和性から、学校現場では生徒が使う自作教材の API として Azure OpenAI が有力な選択肢の一つとなっている。実例として、東京都教育委員会の「都立 AI」は、

256 校・約 16 万人が利用する生成 AI 基盤を Azure OpenAI で構築し、閉域ネットワークやガバナンスを徹底している。もっとも、これを裏づける全国的な調査があるわけではなく、実際には Gemini や Copilot、教育向けの ChatGPT 等も用いられており、どの API を選ぶかは自治体や学校のポリシーに依存する。また、「Azure だから何でも安全」ではない。データの保存・アクセス権限の設計、教育委員会や学校の承認、利用規約の順守は、いずれも別途必要である。

4. 混同されがちな二つの制限

多くの教師が取り違えるのが、「生成 AI の利用制限」と「API で作った教材（作品）の制限」である。両者は別物である。

表 C　「直接利用」と「API 経由の自作教材」の違い

観点	① 生成 AI を直接使う（利用制限）	② API で作った自作教材を使う
制限を決める主体	サービス提供者（各社）。	開発者（教師・学校）＋使う API の規約。
生徒のアカウント	本人のアカウントが必要で、年齢制限が直接かかる。	不要。生徒は教師のアプリを使う。
年齢制限の適用	13 歳・18 歳などが直接適用される。	対象や範囲を開発者が設計する。
別途守るべきこと	規約・保護者同意。	API の規約・データ保護・個人情報・校内の情報セキュリティポリシー。

さらに、これとは別の論点として「生成物（作品）のライセンス」がある。AI が作った文章や画像などを、誰が、どこまで使えるか（成果物として提出してよいか、二次利用や公開は可能か等）は、サービスや API の規約によって異なる。文部科学省のガイドラインも、確認事項として「生成物のライセンスの所在」を明示している。「（アクセスして）使えるか」と「（作った物を）使ってよいか」は、別の問題として押さえる必要がある。

よくある誤解を挙げておく。「生徒は ChatGPT を使えない、だから AI は使えない」――これは誤りで、Azure OpenAI 等で教師が作ったアプリなら、設計次第で利用できる。「環境変数に入れたから鍵は安全」――クライアントに配られる変数は露出しうる。「Firebase の API キーが見えている＝危険」――守りはセキュリティルールであり、キーの露出そのものが問題の本質ではない。「Azure にしたから安心」――データ・権限・承認・規約の確認は依然として必要である。

5. 見落とされがちな運用面 ― 保護者通知・事前指導・情報基礎の授業

技術的に安全でも、教育的・手続き的な備えが欠ければ、適切な運用とはいえない。文部科学省「初等中等教育段階における生成 AI の利活用に関するガイドライン（Ver.2.0）」は、児童生徒に利活用させる際の確認事項として、おおむね次の点を挙げている。

- 生成 AI の性質やメリット・デメリット、情報の真偽を確かめること、最後は自分の判断が大切であることを学ぶ「事前の学習」を行っているか（情報活用能力・情報モラルが十分でない段階で自由に使わせることは望ましくない、とされる）。
- プロンプトに氏名・写真等の個人情報を入力しないよう指導しているか。
- 著作権の侵害につながる使い方をしないよう指導しているか。
- サービス提供者の最新の利用規約を確認・遵守しているか（年齢制限、保護者同意の必要性、生成物のライセンスの所在など）。
- 生成物をそのまま自分の成果物として使わないよう指導しているか。

共通の観点としては、①関係法令・利用規約を守り、開発者の想定範囲内で使う「安全性を考慮した適正利用」、②「教育情報セキュリティポリシーに関するガイドライン」を踏まえた情報セキュリティの確保が求められている。実務としては、授業で生成 AI を使う前に、保護者へ導入の趣旨・使うツール・データの扱いを説明する文書を配り、必要に応じて同意を得ることが望ましい。とりわけ、年齢制限で保護者同意が要るサービスを用いる場合は欠かせない。

ここで注意したいのは、作ることが容易になったからこそ、これらが後回しになりやすいという点である。ツールは短時間で作れても、事前指導・保護者説明・規約確認は手間がかかり、おろそかになりがちである。しかし、この部分が抜けると、技術的には安全でも、教育的・手続き的な安全を欠くことになる。

自作教材を生徒に使わせる前のチェックリスト

□ 秘密の鍵はコードに書かず、シークレット管理に置いたか。

□ 公開範囲は最小か（要ログイン／限定公開／PIN）。権限はサーバー側で担保したか。

□ 生徒の個人情報は最小限か。保存先・権限・保持期間（自動削除）を決めたか。

□ 使う API は学校で認められたもの（例：Azure OpenAI）か。データが学習に使われない設計か。

□ 使うサービス／API の利用規約（年齢・保護者同意・生成物のライセンス）を確認したか。

□ 事前に、生成 AI の特徴・長所短所・個人情報・著作権に関する学習（情報基礎／情報モラル）を行ったか。

□ 保護者への説明・同意（必要な場合）を行ったか。

□ 教育委員会・学校の情報セキュリティポリシーに沿っているか。

6. まとめ

教材を自作できる自由を支えるのは、三つの土台である。第一に、公開・デプロイ環境の仕組みとリスクの理解。第二に、API・利用規約の理解――「使えるか（利用制限）」と「作った物を使ってよいか（生成物のライセンス）」を区別すること。第三に、事前指導と保護者対応。作れることと、安全に運用できることは別である。技術の敷居が下がったいまこそ、この三つを意識的に点検することが、自作教材を真に価値あるものにする。

参考資料

1）文部科学省「初等中等教育段階における生成 AI の利活用に関するガイドライン（Ver.2.0）」（令和 6 年 12 月 26 日）。児童生徒に利活用させる際の確認事項（事前の学習、個人情報・著作権の指導、利用規約の確認〔年齢制限・保護者同意・生成物のライセンスの所在〕等、p.10〜11 ほか）。

2）生成 AI 各社の年齢制限：ChatGPT（13 歳以上、18 歳未満は保護者同意）、Gemini（一般 13 歳以上、学校用アカウントは 18 歳以上）、Copilot（日本の個人は実質 18 歳以上／学校の Microsoft 365 アカウントは 2025 年半ば以降 13 歳以上）、Claude（18 歳以上）。各社利用規約および各種解説記事による。

3）Azure OpenAI Service の位置づけ（入力データを学習に用いない、開発者が年齢・内容を制御）および Microsoft Learn・Microsoft 教育向け資料。東京都教育委員会「都立 AI」（256 校・約 16 万人、Azure OpenAI 基盤）：Microsoft 導入事例による。

4）Firebase の API キーとセキュリティルール：Firebase 公式ドキュメント（セキュリティチェックリスト、基本的なセキュリティルール）および各技術解説。

5）Google Apps Script のデプロイ設定（実行するユーザー／アクセスできるユーザー）：Google for Developers 公式ドキュメントおよび各解説。

6）Streamlit のシークレット管理：Streamlit 公式ドキュメント（Secrets management）。GitHub のシークレットスキャン等：各解説。

※年齢制限・利用規約・各サービスの仕様は改訂されうる。導入時には各社の最新の規約と、所属自治体・学校の情報セキュリティポリシーを必ず確認されたい。